

Design-Weighted LCA: Data and Model Selection

Institutional trust in Mexico: LAPOP AmericasBarometer 2023

Kevin Linares

2026-07-25

Purpose

Presents the data and the recode audit, searches over K with the design-weighted pseudo-likelihood, and, once `cfg$K_force` is set, produces the fit diagnostics and the design-based intervals, then saves the fit for the segments script. Nothing here is interpretive.

The data

Grey is the missingness the analysis rule removes. The recode audit shows what every source label became; anything that failed to match would have stopped the config.

```
plot_item_stack(survey_dat_full, items, "Items as read, with missingness")
```

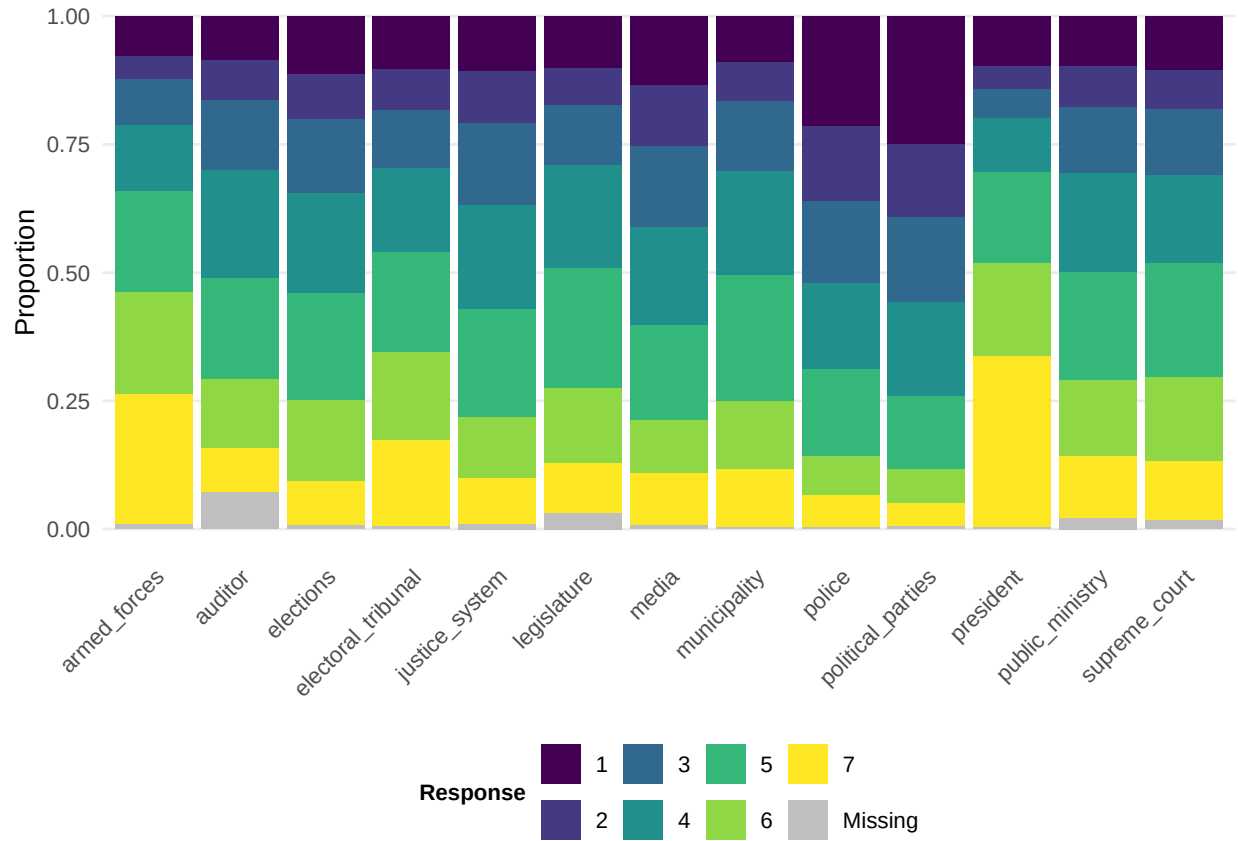


Figure 1: All respondents, all items, missingness shown explicitly (grey).

```
cat(sprintf("%d of %d respondents enter the analysis.\n",
           nrow(cfg$data), nrow(survey_dat_full)))
```

1454 of 1622 respondents enter the analysis.

```
lca_table(recode_audit,
          col.names = c("Variable", "Source label", "Recoded to", "n"),
          widths = c("6em", "21em", "9em", "3em"),
          caption = "Recode audit: how each source label was mapped.")
```

Table 1: Recode audit: how each source label was mapped.

Variable	Source label	Recoded to	n
age_cat	29	18-29	70
age_cat	18	18-29	69
age_cat	28	18-29	67
age_cat	50	45-59	63
age_cat	27	18-29	53
age_cat	48	45-59	49
age_cat	19	18-29	42

Table 1: Recode audit: how each source label was mapped. *(continued)*

Variable	Source label	Recoded to	n
age_cat	26	18-29	41
age_cat	52	45-59	41
age_cat	49	45-59	39
age_cat	32	30-44	37
age_cat	25	18-29	36
age_cat	24	18-29	33
age_cat	40	30-44	33
age_cat	31	30-44	32
age_cat	35	30-44	31
age_cat	51	45-59	31
age_cat	30	30-44	30
age_cat	22	18-29	29
age_cat	42	30-44	29
age_cat	53	45-59	29
age_cat	20	18-29	28
age_cat	23	18-29	27
age_cat	43	30-44	27
age_cat	33	30-44	25
age_cat	37	30-44	25
age_cat	39	30-44	25
age_cat	44	30-44	25
age_cat	46	45-59	25
age_cat	54	45-59	25
age_cat	21	18-29	24
age_cat	36	30-44	24
age_cat	45	45-59	23
age_cat	38	30-44	22
age_cat	60	60+	22
age_cat	47	45-59	21
age_cat	55	45-59	21
age_cat	56	45-59	21
age_cat	34	30-44	20
age_cat	72	60+	20
age_cat	63	60+	19
age_cat	70	60+	19
age_cat	61	60+	18
age_cat	58	45-59	17
age_cat	62	60+	17
age_cat	68	60+	17
age_cat	57	45-59	15
age_cat	66	60+	15
age_cat	41	30-44	14
age_cat	65	60+	14
age_cat	69	60+	14
age_cat	59	45-59	13
age_cat	64	60+	13
age_cat	73	60+	13
age_cat	67	60+	11
age_cat	78	60+	7
age_cat	71	60+	6
age_cat	77	60+	6
age_cat	74	60+	5
age_cat	76	60+	5
age_cat	82	60+	4
age_cat	85	60+	4
age_cat	75	60+	3
age_cat	79	60+	3
age_cat	80	60+	3
age_cat	81	60+	3
age_cat	84	60+	3
age_cat	83	60+	2

Table 1: Recode audit: how each source label was mapped. *(continued)*

Variable	Source label	Recoded to	n
age_cat	87	60+	2
age_cat	86	60+	1
age_cat	92	60+	1
age_cat	94	60+	1
education	Secundaria o Educación Media Superior/Bachillerato/Preparatoria/Profesional Técnico completa	Secondary	775
education	Universitaria, superior no universitaria o técnico universitario completa	Tertiary	218
education	Primaria completa	Primary	193
education	Secundaria o Educación Media Superior/Bachillerato/Preparatoria/Profesional Técnico incompleta	Secondary	168
education	Primaria incompleta	Primary	133
education	Universitaria, superior no universitaria o técnico universitario incompleta	Tertiary	95
education	Ninguna	None	38
education	NR	Missing	2
employment	Trabajando?	Employed	787
employment	Se dedica a los quehaceres de su hogar?	Homemaker	447
employment	Es estudiante?	Student	95
employment	Está jubilado, pensionado o incapacitado permanentemente para trabajar?	Retired	95
employment	Está buscando trabajo activamente?	Unemployed	73
employment	No está trabajando en este momento pero tiene trabajo?	Employed	70
employment	No trabaja y no está buscando trabajo?	Not in labor force	50
employment	NR	Missing	3
employment	DK	Missing	2
prez_rating	Bueno	Good	778
prez_rating	Muy bueno	Very good	394
prez_rating	Ni bueno, ni malo (regular)	Neither	335
prez_rating	Malo	Bad	66
prez_rating	Muy malo (pésimo)	Very bad	44
prez_rating	DK	Missing	4
prez_rating	NR	Missing	1
satis_demo	Satisfecho(a)	Satisfied	851
satis_demo	Insatisfecho(a)	Dissatisfied	529
satis_demo	Muy insatisfecho(a)	Very dissatisfied	110
satis_demo	Muy satisfecho(a)	Very satisfied	103
satis_demo	DK	Missing	25
satis_demo	NR	Missing	4
sex	Mujer/femenino	Female	830
sex	Hombre/masculino	Male	787
sex	DK	Missing	3
sex	NR	Missing	2
urban	Urbano	Urban	1294
urban	Rural	Rural	328

```
dictionary |>
  mutate(responses = map_chr(responses, \(x) paste(x, collapse = " | "))) |>
  lca_table(col.names = c("Analysis name", "Source variable",
    "Survey question", "Response labels"),
    widths = c("8em", "5em", "16em", "14em"),
    caption = "Item dictionary: analysis names, source variables, question wording, and")
```

Table 2: Item dictionary: analysis names, source variables, question wording, and the response labels the segment labeler reads.

Analysis name	Source variable	Survey question	Response labels
justice_system	b10a	Confianza en el sistema de justicia	Nada 2 3 4 5 6 Mucho
electoral_tribunal	b11	Confianza en el Tribunal Supremo Electoral	Nada 2 3 4 5 6 Mucho
armed_forces	b12	Confianza en las fuerzas armadas	Nada 2 3 4 5 6 Mucho
legislature	b13	Confianza en la legislatura	Nada 2 3 4 5 6 Mucho
public_ministry	b15	Confianza en el Ministerio Público	Nada 2 3 4 5 6 Mucho
police	b18	Confianza en la policía nacional	Nada 2 3 4 5 6 Mucho
auditor	b19	Confianza en la Auditoría Superior de la Federación	Nada 2 3 4 5 6 Mucho
political_parties	b21	Confianza en los partidos políticos	Nada 2 3 4 5 6 Mucho
president	b21a	Confianza en el presidente	Nada 2 3 4 5 6 Mucho
supreme_court	b31	Confianza en el tribunal supremo	Nada 2 3 4 5 6 Mucho
municipality	b32	Confianza en su municipalidad	Nada 2 3 4 5 6 Mucho
media	b37	Confianza en los medios de comunicación	Nada 2 3 4 5 6 Mucho
elections	b47a	Confianza en las elecciones	Nada 2 3 4 5 6 Mucho

```

dat_prepared <- cfg$data
w_vec <- dat_prepared[[cfg$weight]]
n_obs <- nrow(dat_prepared)
n_eff <- sum(w_vec)^2 / sum(w_vec^2)
scale_ic <- n_obs / sum(w_vec)

cat(sprintf("Weights: CV = %.3f, unequal-weighting effect = %.2f, effective n = %.0f of %d.\n"
            sd(w_vec) / mean(w_vec), 1 + (sd(w_vec) / mean(w_vec))^2, n_eff, n_obs))

```

Weights: CV = 0.000, unequal-weighting effect = 1.00, effective n = 1454 of 1454.

The unequal-weighting effect is what design weighting costs in variance. When it is close to 1, the weighted and unweighted fits will differ little and the gap reported later should be read as noise rather than correction.

Searching over K

Every candidate K is fit from `cfg$n_starts` seeded starts. Seeds are generated in this session and passed to the workers, so sequential and parallel plans give identical results. BIC uses the achieved sample size and `BIC_eff` the effective sample size, which is the design-aware penalty; where they disagree, weight `BIC_eff` more heavily on a clustered sample.

```

enum <- future_map(cfg$K_range, function(K) {
  fit <- fit_lca(dat_prepared, w_vec, cats, cfg$items, K, start_seeds(cfg, K))
  ll_n <- fit$ll * scale_ic
  d <- df_k(K, cats)
  tibble(K = K, logLik = ll_n, df = d,

```

```

    AIC = -2 * ll_n + 2 * d,
    BIC = -2 * ll_n + d * log(n_obs),
    BIC_eff = -2 * ll_n + d * log(n_eff),
    entropy = entropy_R2(fit$post, w_vec, K),
    converged = fit$converged)
}, .options = furrr_options(seed = NULL,
  packages = c("matrixStats", "clue", "purrr", "dplyr", "tidyr"))) |>
list_rbind()

enum |>
  mutate(across(c(logLik, AIC, BIC, BIC_eff), \(x) round(x, 1)),
    entropy = round(entropy, 3)) |>
  lca_table(align = "r",
    caption = "Weighted fit statistics by number of segments. Nothing is selected here

```

Table 3: Weighted fit statistics by number of segments. Nothing is selected here. Read BIC_eff and entropy together, then set `cfg$K_force`.

K	logLik	df	AIC	BIC	BIC_eff	entropy	converged
2	-32115.8	157	64545.7	65375.0	65375.0	0.903	TRUE
3	-30686.4	236	61844.8	63091.4	63091.4	0.913	TRUE
4	-29945.9	315	60521.9	62185.7	62185.7	0.921	TRUE
5	-29526.9	394	59841.8	61922.9	61922.9	0.908	TRUE
6	-29223.8	473	59393.6	61892.0	61892.0	0.895	TRUE
7	-29004.9	552	59113.7	62029.4	62029.4	0.893	TRUE
8	-28855.3	631	58972.7	62305.6	62305.6	0.890	TRUE
9	-28707.5	710	58835.1	62585.3	62585.3	0.887	TRUE
10	-28588.4	789	58754.9	62922.4	62922.4	0.896	TRUE

```

enum |>
  select(K, BIC, BIC_eff, entropy) |>
  pivot_longer(-K, names_to = "criterion", values_to = "value") |>
  mutate(criterion = recode(criterion, BIC = "BIC (n)",
    BIC_eff = "BIC (effective n)",
    entropy = "Relative entropy")) |>

ggplot(aes(K, value)) +
  geom_line(linewidth = 0.6, color = viridisLite::viridis(1, begin = 0.4)) +
  geom_point(size = 2.4, color = viridisLite::viridis(1, begin = 0.4)) +
  facet_wrap(~ criterion, scales = "free_y") +
  scale_x_continuous(breaks = cfg$K_range) +
  labs(x = "Number of segments (K)", y = NULL) +
  theme_lca()

```

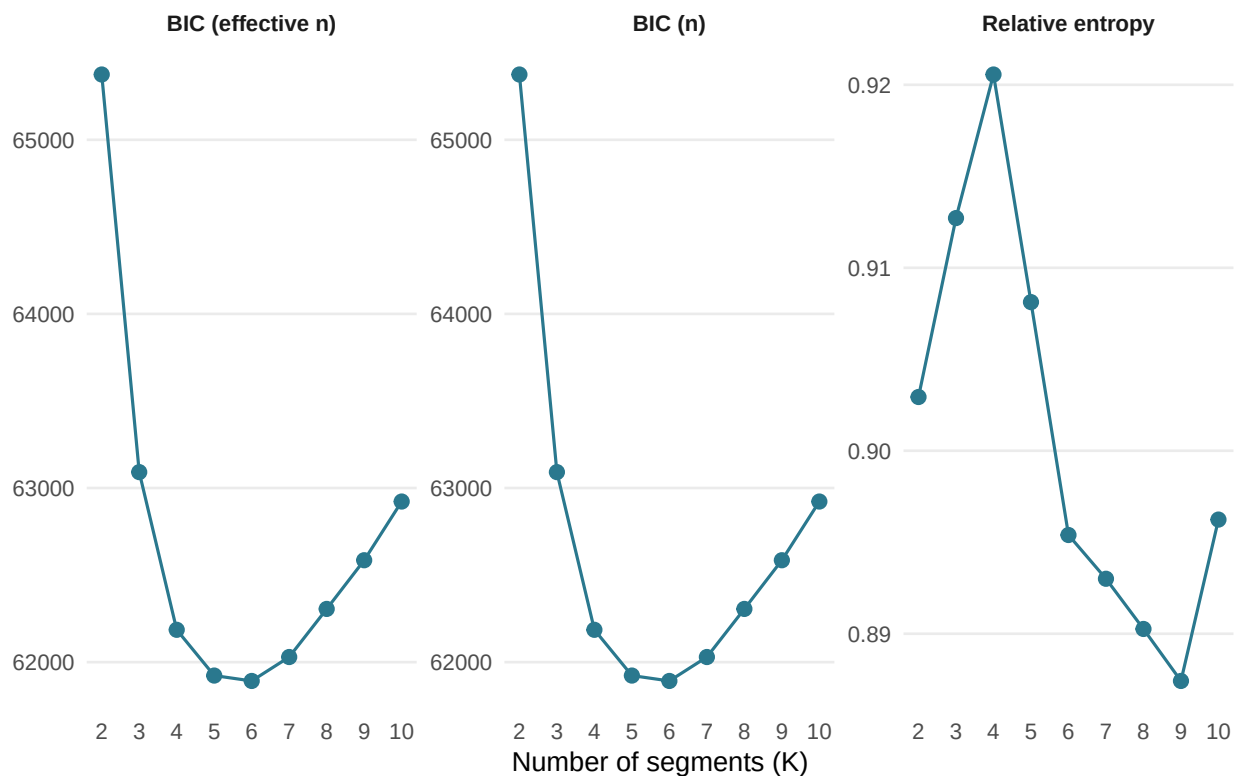


Figure 2: Enumeration diagnostics across K. BIC_eff penalizes on the effective sample size; entropy indexes classification sharpness. Neither decides alone.

```
if (!K_ready) cat("**`cfg$K_force` is `NULL`.** Review the evidence above, set a candidate K in
```

The chosen model

```
K_star <- as.integer(cfg$K_force)

fit_w <- fit_lca(dat_prepared, w_vec, cats, cfg$items, K_star,
               start_seeds(cfg, K_star), maxit = 5000L, tol = 1e-10)
fit_u <- align_to(fit_lca(dat_prepared, rep(1, n_obs), cats, cfg$items, K_star,
               start_seeds(cfg, K_star), maxit = 5000L, tol = 1e-10),
               fit_w)

inp_full <- make_inputs(dat_prepared, cfg$items, cats)
post_w <- posterior_of(fit_w$pi, fit_w$rho, inp_full$Y)
colnames(post_w) <- paste0("post_segment", seq_len(K_star))
modal_int <- max.col(post_w, ties.method = "first")

dat_prepared <- bind_cols(dat_prepared, as_tibble(post_w))
```

```
cat(sprintf(paste0("Fitted K = %d on %d cases. Converged: %s.\n",
                  "Weighted log-likelihood (sum-to-n scale): %.1f | weighted entropy: %.3f\n",
                  K_star, n_obs,
                  if (isTRUE(fit_w$converged)) "yes" else "NO, inspect before trusting",
                  fit_w$ll * scale_ic, entropy_R2(post_w, w_vec, K_star)))
```

Fitted K = 5 on 1454 cases. Converged: yes.

Weighted log-likelihood (sum-to-n scale): -29526.9 | weighted entropy: 0.908

```
tibble(Segment = paste("Segment", seq_len(K_star)),
       `Weighted prevalence` = round(fit_w$pi, 3),
       `Unweighted prevalence` = round(fit_u$pi, 3),
       `Modal-assignment share` = round(as.numeric(prop.table(
         table(factor(modal_int, levels = seq_len(K_star))))) , 3)) |>
lca_table(align = "lrrr", widths = c("12em", "11em", "11em", "11em"),
         caption = "Segment prevalences at the chosen K. The modal-assignment share differs
```

Table 4: Segment prevalences at the chosen K. The modal-assignment share differs from the model prevalence whenever classification is uncertain.

Segment	Weighted prevalence	Unweighted prevalence	Modal-assignment share
Segment 1	0.341	0.341	0.343
Segment 2	0.207	0.207	0.207
Segment 3	0.107	0.107	0.107
Segment 4	0.234	0.234	0.233
Segment 5	0.112	0.112	0.111

Which items carry the segmentation

```
item_discrim <- item_discrimination(fit_w, cfg$items)

item_discrim |>
  mutate(discrimination = round(discrimination, 3)) |>
  lca_table(col.names = c("Item", "Discrimination"), align = "lr",
           caption = "Item discrimination: mean over segment pairs of the total variation dis
```

Table 5: Item discrimination: mean over segment pairs of the total variation distance between their response distributions. Bounded in $[0, 1]$. Zero means every segment answers alike.

Item	Discrimination
justice_system	0.757
public_ministry	0.706
auditor	0.699
supreme_court	0.689

Table 5: Item discrimination: mean over segment pairs of the total variation distance between their response distributions. Bounded in $[0, 1]$. Zero means every segment answers alike. *(continued)*

Item	Discrimination
legislature	0.664
elections	0.658
political_parties	0.631
electoral_tribunal	0.621
municipality	0.620
police	0.585
media	0.490
armed_forces	0.485
president	0.457

```
item_discrim |>
  mutate(item = factor(item, levels = rev(item))) |>
  ggplot(aes(discrimination, item)) +
  geom_col(fill = viridisLite::viridis(1, begin = 0.45), width = 0.7) +
  geom_vline(xintercept = 0.1, linetype = "dashed", color = "grey55") +
  scale_x_continuous(limits = c(0, 1)) +
  labs(x = "Discrimination (mean pairwise total variation distance)", y = NULL) +
  theme_lca()
```

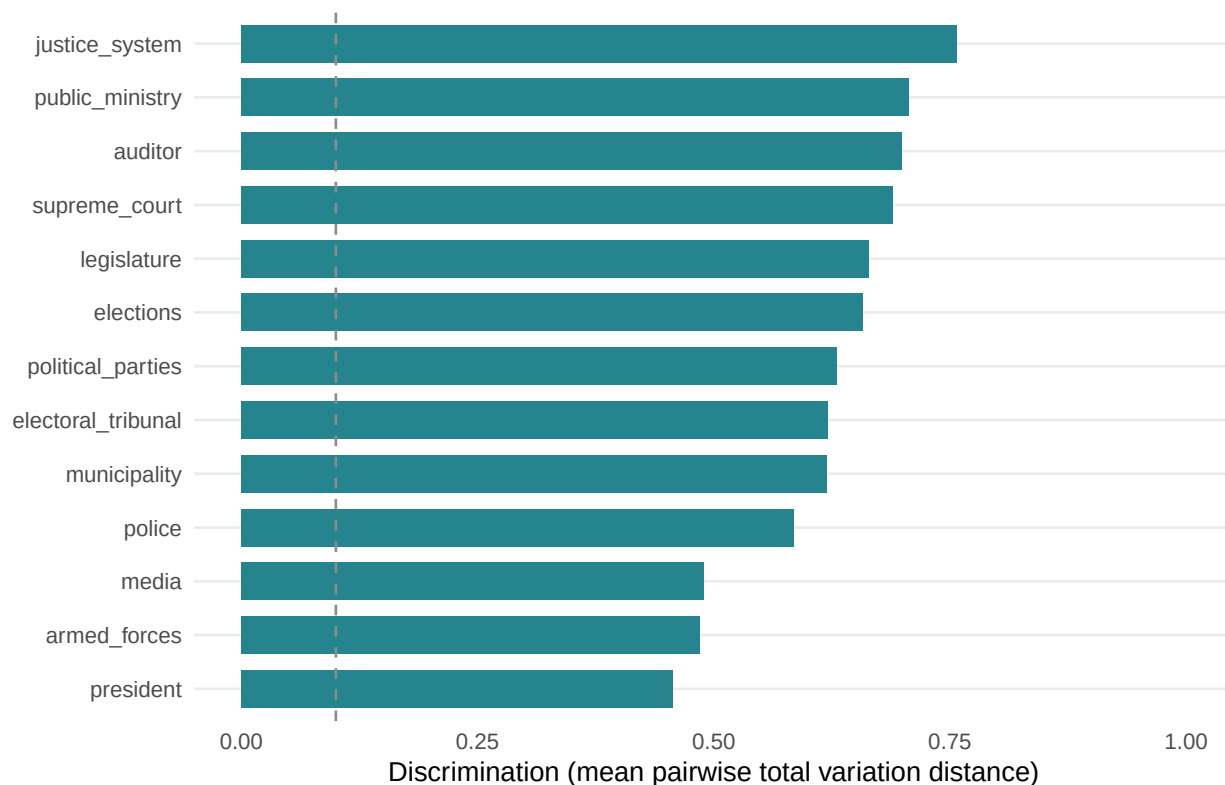


Figure 3: Item discrimination. Low bars are candidates for removal in a sensitivity refit.

```

# Does each item earn its segment-dependent parameters? Compare the fitted model
# against one where item j is segment-independent. Under local independence that
# restricted model factorises into an LCA on the other items times item j's
# marginal, so the two BICs are comparable. Negative delta_bic means BIC prefers
# dropping the item's segment-dependence.
ll_full <- fit_w$ll * scale_ic
df_full <- df_k(K_star, cats)

marginal_ll <- function(item) {
  y <- as.integer(dat_prepared[[item]])
  p <- as.numeric(xtabs(w_vec ~ y))
  sum(w_vec * log((p / sum(p))[y]))
}

item_screen <- item_discrim |>
  mutate(delta_bic = future_map_dbl(item, function(j) {
    keep_items <- setdiff(cfg$items, j)
    fit_wo <- fit_lca(dat_prepared, w_vec, cats[keep_items], keep_items, K_star,
                      start_seeds(cfg, K_star))
    ll_restr <- (fit_wo$ll + marginal_ll(j)) * scale_ic
    df_restr <- df_k(K_star, cats[keep_items]) + (cats[[j]] - 1)
    (-2 * ll_restr + df_restr * log(n_eff)) - (-2 * ll_full + df_full * log(n_eff))
  }), .options = furrr_options(seed = NULL,
    packages = c("matrixStats", "clue", "purrr", "dplyr", "tidyr")) |>
  arrange(delta_bic)

item_screen |>
  mutate(discrimination = round(discrimination, 3), delta_bic = round(delta_bic, 1)) |>
  lca_table(align = "lrr", col.names = c("Item", "Discrimination", "Delta BIC"),
    widths = c("14em", "10em", "8em"),
    caption = "Indicator screen, most droppable first. Delta BIC is BIC(item made segm

```

Table 6: Indicator screen, most droppable first. Delta BIC is BIC(item made segment-independent) minus BIC(full model), on the effective sample size. Negative flags the item. This is a ranking under a pseudo-likelihood, not a Bayes factor, so the usual BIC-difference thresholds do not apply.

Item	Discrimination	Delta BIC
president	0.457	336.9
armed_forces	0.485	437.6
media	0.490	501.3
police	0.585	755.6
electoral_tribunal	0.621	860.8
political_parties	0.631	895.8
municipality	0.620	908.3
elections	0.658	1014.9
legislature	0.664	1016.5
supreme_court	0.689	1154.2
public_ministry	0.706	1190.9
auditor	0.699	1227.7

Table 6: Indicator screen, most droppable first. Delta BIC is BIC(item made segment-independent) minus BIC(full model), on the effective sample size. Negative flags the item. This is a ranking under a pseudo-likelihood, not a Bayes factor, so the usual BIC-difference thresholds do not apply. *(continued)*

Item	Discrimination	Delta BIC
justice_system	0.757	1460.1

Local independence: bivariate residuals

```
bvr_tbl <- bvr_pairs(dat_prepared, w_vec, cfg$items, fit_w)

bvr_tbl |>
  mutate(bvr = round(bvr, 3)) |>
  slice_head(n = 10) |>
  lca_table(align = "llr", col.names = c("Item A", "Item B", "BVR"),
            widths = c("14em", "14em", "6em"),
            caption = "Largest bivariate residuals. BVR is the total variation distance between
```

Table 7: Largest bivariate residuals. BVR is the total variation distance between the weighted observed two-way table and the model-implied table, bounded in $[0, 1]$ and zero under exact local independence. No valid reference distribution applies here, so this ranks rather than tests.

Item A	Item B	BVR
public_ministry	auditor	0.163
armed_forces	legislature	0.148
justice_system	supreme_court	0.120
police	political_parties	0.106
police	municipality	0.104
president	elections	0.100
electoral_tribunal	public_ministry	0.099
auditor	supreme_court	0.099
president	supreme_court	0.095
armed_forces	president	0.094

```
bvr_tbl |>
  mutate(item_a = factor(item_a, levels = cfg$items),
         item_b = factor(item_b, levels = cfg$items)) |>
  ggplot(aes(item_a, item_b, fill = bvr)) +
  geom_tile() +
  geom_text(aes(label = sprintf("%.2f", bvr)), size = 2.0) +
  scale_fill_viridis_c(name = "BVR", direction = -1) +
  labs(x = NULL, y = NULL) +
  theme_lca() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

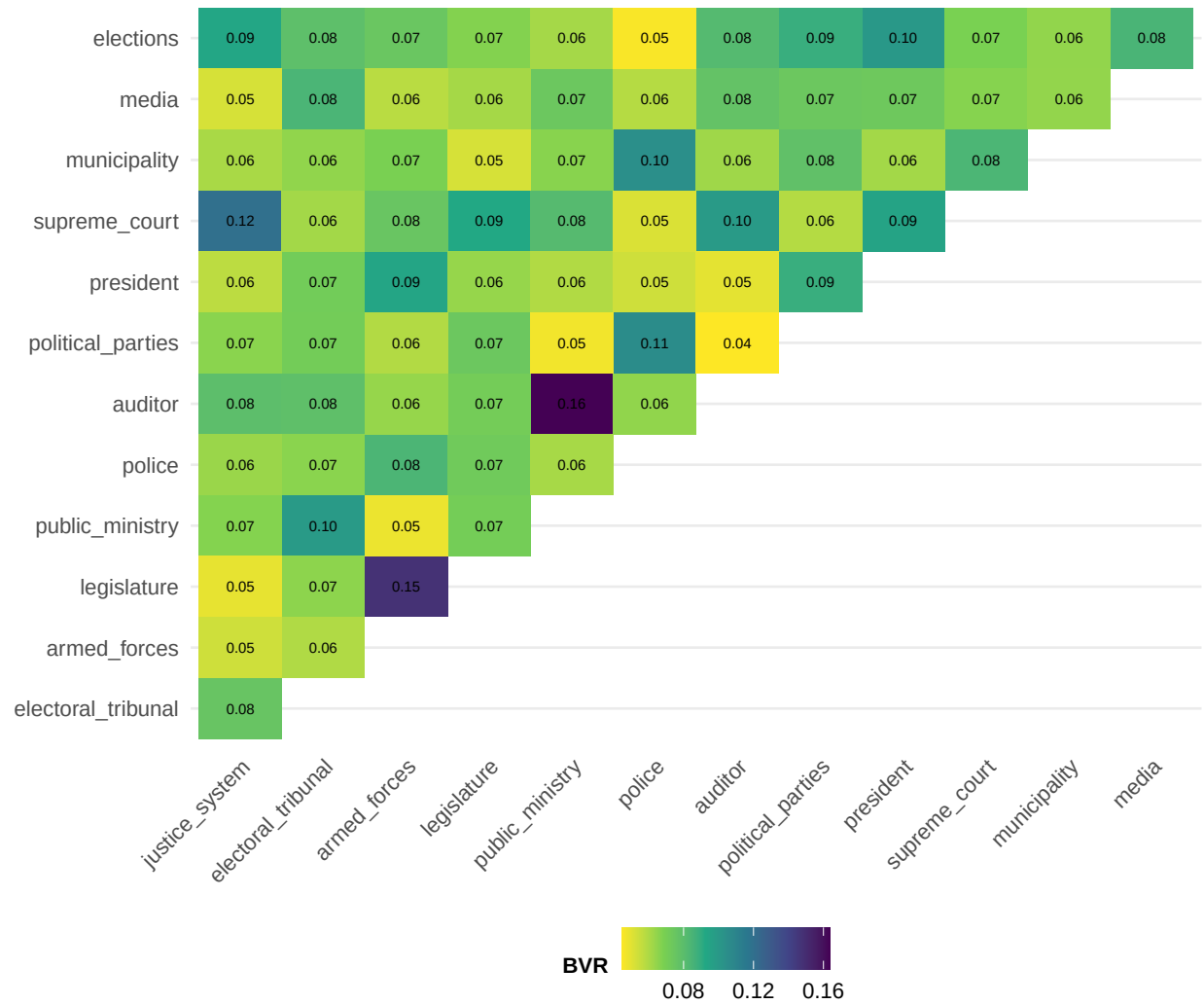


Figure 4: Bivariate residuals as total variation distance. Darker cells are pairs whose association the segments explain least well. Persistent dark pairs argue for dropping or merging an item, or trying $K+1$.

Design-based confidence intervals

```
design <- build_rep_design(dat_prepared, cfg)
des <- design$des
rep_des <- design$rep_des
design_df <- degf(rep_des)

cat(sprintf("Design: %d replicates, %d degrees of freedom.\n",
            ncol(rep_des$repweights), design_df))
```

Design: 130 replicates, 126 degrees of freedom.

```

# Parameter bookkeeping, in the order flatten_fit() produces.
param_meta <- bind_rows(
  tibble(kind = "pi", item = NA_character_, category = NA_integer_,
    segment = seq_len(K_star)),
  imap(cats, function(Cj, item_name)
    expand_grid(segment = seq_len(K_star), category = seq_len(Cj)) |>
      transmute(kind = "rho", item = item_name, category, segment)) |>
    list_rbind())

flatten_fit <- function(fit) c(fit$pi, unlist(map(fit$rho, as.vector)))

# Refit on each replicate, warm-started at the full-sample solution and aligned
# to it, at the same tolerance as the point estimate.
theta_fun <- function(w_rep) {
  fit <- em_run(inp_full$Y, inp_full$OH, cats, w_rep, K_star,
    init = fit_w[c("pi", "rho")], maxit = 1000L, tol = 1e-10)
  flatten_fit(aligned_to(fit, fit_w))
}

est_vec <- flatten_fit(fit_w)
se_vec <- sqrt(diag(replicate_variance(rep_des, theta_fun, est_vec)))

```

```

# Intervals on the logit scale by the delta method, back-transformed, with the
# design t critical value. Estimates at a boundary have no meaningful logit SE
# and fall back to Wald truncated to [0, 1].
crit <- qt(0.975, df = design_df)

ci_tbl <- param_meta |>
  mutate(estimate = as.numeric(est_vec), se = as.numeric(se_vec),
    boundary = estimate < 1e-3 | estimate > 1 - 1e-3,
    p_c = pmin(pmax(estimate, 1e-6), 1 - 1e-6),
    gse = se / (p_c * (1 - p_c)),
    lo = if_else(boundary, pmax(0, estimate - crit * se),
      plogis(qlogis(p_c) - crit * gse)),
    hi = if_else(boundary, pmin(1, estimate + crit * se),
      plogis(qlogis(p_c) + crit * gse))) |>
  select(-p_c, -gse)

```

Unweighted benchmark

At unit weights the weighted EM and poLCA maximize the same function, so any gap beyond optimizer tolerance is an implementation problem rather than a modeling result.

```

polca_df <- dat_prepared |> select(all_of(cfg$items)) |> as.data.frame()
polca_f <- as.formula(paste0("cbind(", paste(cfg$items, collapse = ", "), ") ~ 1"))

```

```

set.seed(cfg$seed + 7L)
pl <- poLCA(polca_f, data = polca_df, nclass = K_star, nrep = 10,
            maxiter = 5000, na.rm = TRUE, verbose = FALSE)
pl_aligned <- align_to(list(pi = as.numeric(pl$P), rho = map(pl$probs, t),
                           post = pl$posterior), fit_w)

# Third candidate: our EM warm-started at poLCA's solution, so the comparison is
# between optima rather than stopping rules.
fit_from_pl <- em_run(inp_full$Y, inp_full$OH, cats, rep(1, n_obs), K_star,
                     init = list(pi = pl_aligned$pi, rho = pl_aligned$rho),
                     maxit = 5000L, tol = 1e-10)
fit_unit <- if (fit_from_pl$ll > fit_u$ll) align_to(fit_from_pl, fit_w) else fit_u

validation <- tibble(
  max_pi_gap = max(abs(fit_unit$pi - pl_aligned$pi)),
  max_rho_gap = max(map2_dbl(fit_unit$rho, pl_aligned$rho, \(a, b) max(abs(a - b))))
validation$match <- max(validation$max_pi_gap, validation$max_rho_gap) < 0.005

cat(sprintf(paste0("VALIDATION (unit weights, our EM vs poLCA):\n",
                   "  max |prevalence gap|           = %.5f\n",
                   "  max |conditional-probability gap| = %.5f\n",
                   "  log-likelihood ours = %.4f | poLCA = %.4f\n  -> %s\n"),
          validation$max_pi_gap, validation$max_rho_gap, fit_unit$ll, pl$llik,
          if (validation$match) "MATCH: implementations agree."
          else "MISMATCH: investigate before trusting the weighted fit.))

```

```

VALIDATION (unit weights, our EM vs poLCA):
  max |prevalence gap|           = 0.00000
  max |conditional-probability gap| = 0.00000
  log-likelihood ours = -29526.8844 | poLCA = -29526.8844
  -> MATCH: implementations agree.

```

```

saveRDS(list(fit_w = fit_w, pl_aligned = pl_aligned, validation = validation,
             dat_prepared = dat_prepared, post_w = post_w, modal_int = modal_int,
             ci_tbl = ci_tbl, bvr_tbl = bvr_tbl, enum = enum,
             w_vec = w_vec, n_eff = n_eff, K = K_star),
         file.path(cfg$out_dir, "model_fit.rds"))
cat("Saved model_fit.rds for K =", K_star, "to", cfg$out_dir, "\n")

```

Saved model_fit.rds for K = 5 to D:/repos/weighted_and_biased/output/mexico